
AIR — Agentic Incident Response

Ship an AI incident commander that investigates at machine speed and acts only when you say go.

1. Executive Summary

AIR (Agentic Incident Response) reframes incident management from a human-driven, tool-assisted process into an AI-driven, human-governed one. Rather than a system of record that engineers update by hand, AIR is designed as an **Incident Operating System**: a standing set of specialized AI agents that investigate, diagnose, and propose remediation continuously, while every state-changing action still passes through explicit human approval and policy enforcement.

The core bet is that most incident-response time is spent on repeatable investigative work — pulling logs, correlating metrics, checking recent deployments, searching prior incidents — that agents can do in parallel and in seconds. AIR keeps humans as the decision-makers on root cause, remediation, and communication, but removes them from the manual-lookup loop. The result is intended to compress Mean Time to Resolution (MTTR) while producing a stronger audit trail than manual processes typically generate.

This document specifies the product vision, system architecture, multi-agent design, data model, APIs, technology stack, and phased delivery plan for engineering teams building AIR.

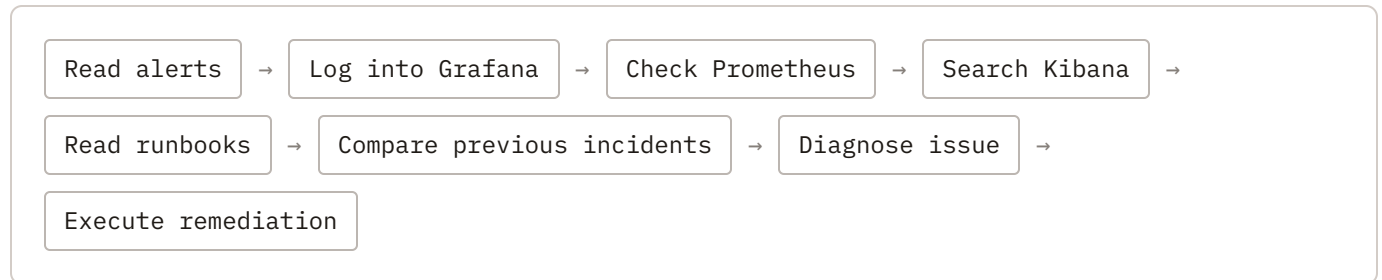
2. Why & Problem Statement

Why

Build an AI-powered Incident Resolution platform that assists SREs and Support Engineers in diagnosing production incidents, recommending root causes, generating remediation steps, and reducing Mean Time to Resolution (MTTR).

Problem to Solve

Current incident management is largely manual. Example — engineers need to:



This process is slow and heavily depends on senior engineers.

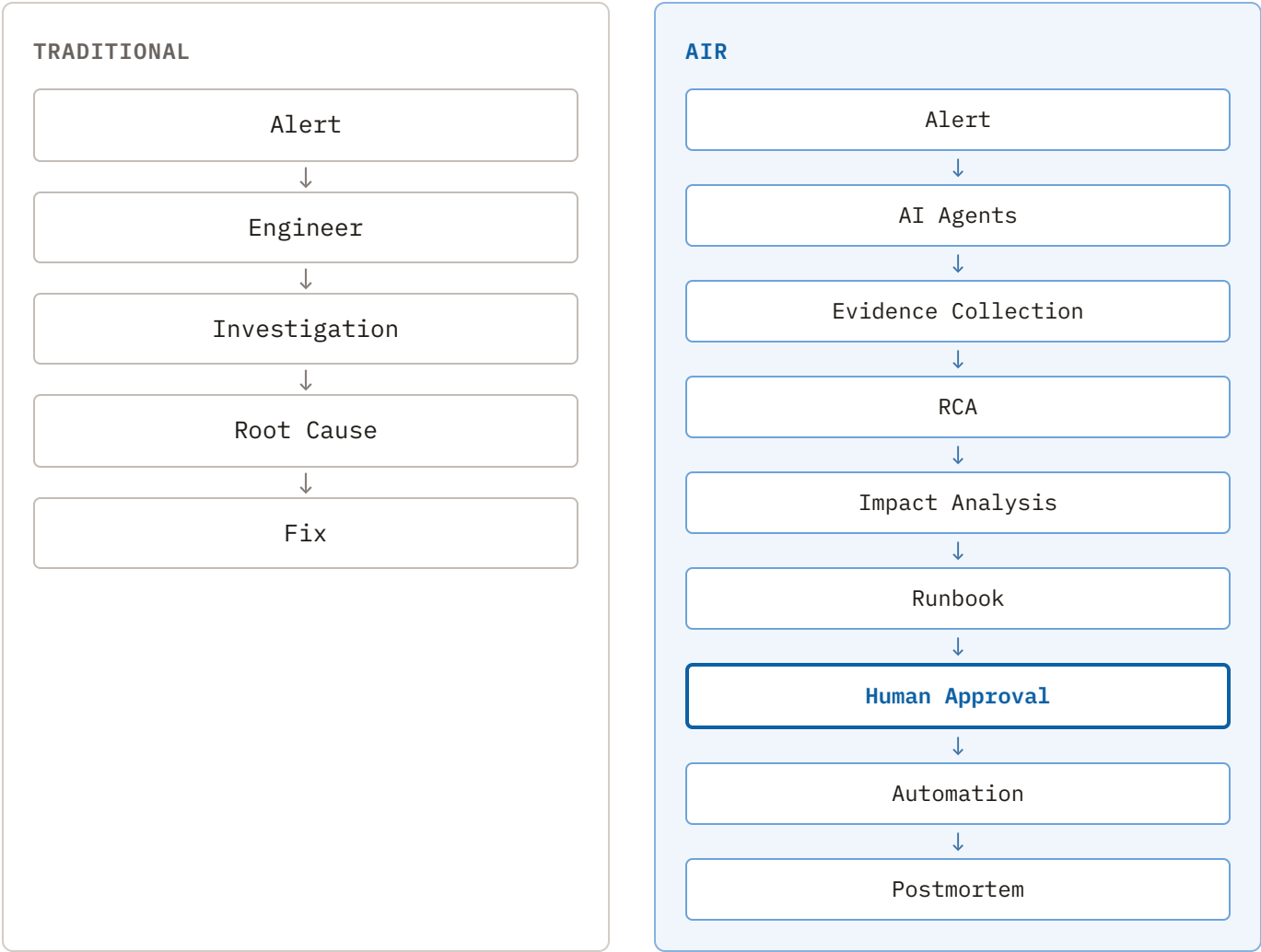
Goals

Business Goals

- Reduce MTTR by 50%
- Reduce false alerts
- Improve SRE productivity
- Reduce operational cost
- Improve SLA compliance

3. Product Vision

Traditional incident response is a linear, human-executed chain: an alert fires, an engineer is paged, they investigate manually, form a root cause hypothesis, and apply a fix. AIR inserts a layer of specialized agents between the alert and the human, so that by the time a person is engaged, evidence, root cause candidates, impact analysis, and a remediation plan already exist and only need review.

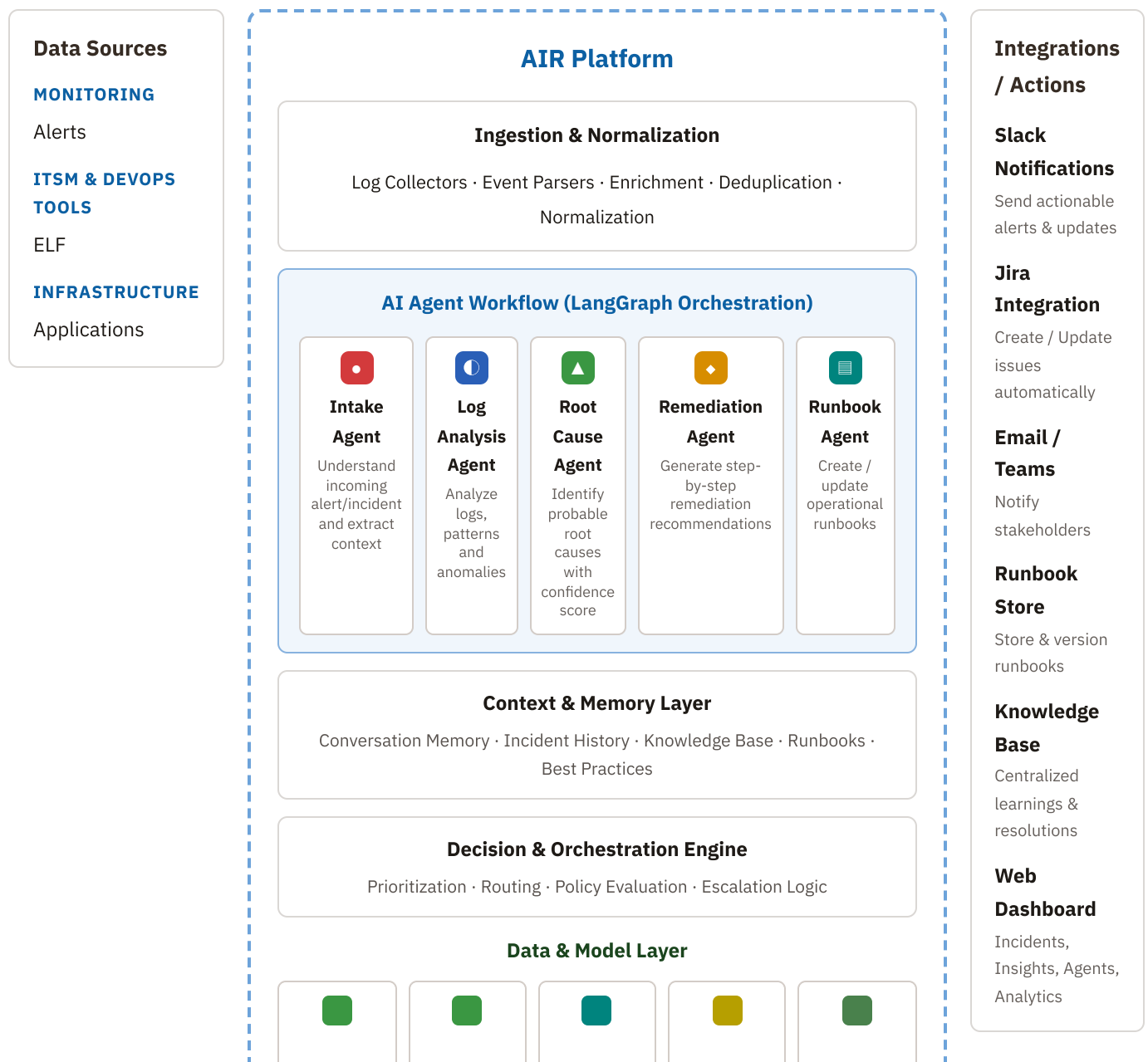


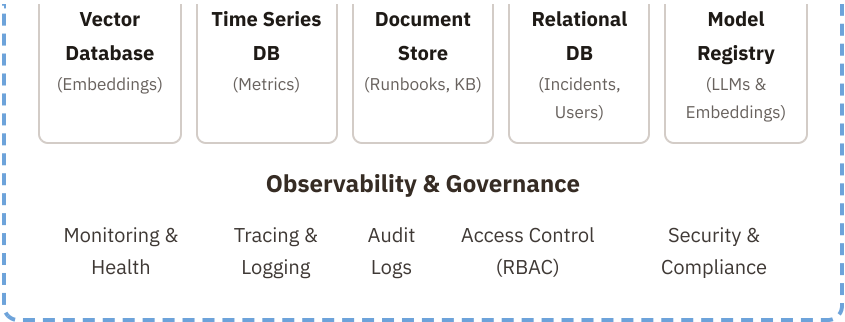
AIR functions as the organization's AI Incident Commander — orchestrating investigation and remediation while keeping a human approval gate before any state-changing action.



AI-Powered Operations Platform

From Signals to Action. Faster Resolution. Lower MTTR.





5. Core Components

Component	Responsibility
Alert Service	Receives alerts from monitoring systems
Incident Service	Creates and manages incidents
AI Orchestrator	Coordinates specialized AI agents
Context Service	Builds operational context
Knowledge Service	RAG over runbooks and historical incidents
RCA Service	Determines probable root causes
Impact Service	Estimates blast radius and business impact
Automation Service	Executes approved remediation
Communication Service	Slack, Teams, Email, Jira integration
Timeline Service	Captures incident chronology
Postmortem Service	Generates AI-assisted retrospectives
Audit Service	Records every AI and human action
Policy Engine	RBAC, approvals, and governance

6. AI Multi-Agent Architecture

Each agent under the Incident Commander owns one investigative or operational concern and hands its output to the next stage. Agents run independently and in parallel where possible; the chain below shows each agent's internal steps.

Investigation Agent

Log Analysis → Metrics Analysis → Trace Analysis → Deployment Analysis → Dependency Analysis

RCA Agent

Hypothesis Ranking → Confidence Score → Evidence

Impact Agent

Affected Services → Customers → Revenue → SLA

Runbook Agent

Find Playbook → Generate Steps → Automation Plan

Communication Agent

Slack → Teams → Email → Status Page

Automation Agent

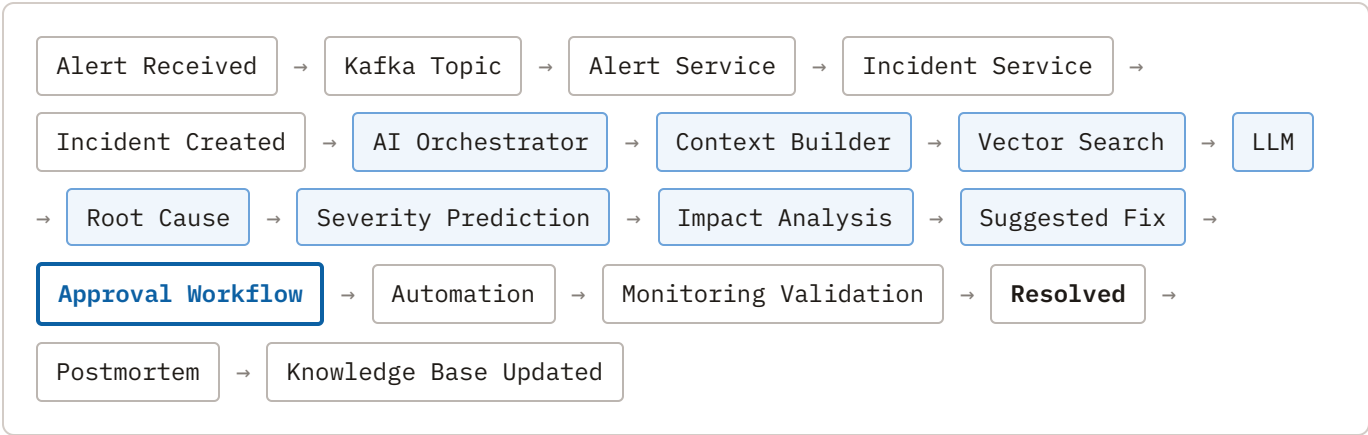
Rollback → Restart → Scale → Verification

Postmortem Agent

Timeline → Lessons → Action Items

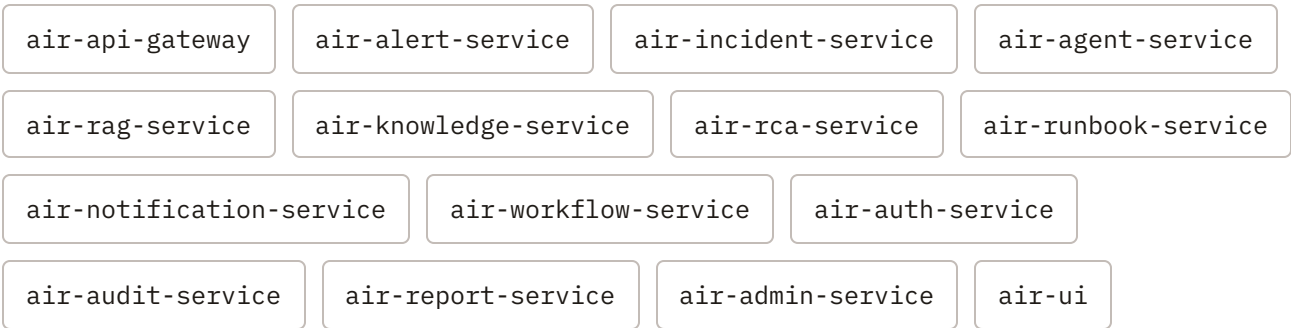
7. End-to-End Data Flow

A single alert's journey from ingestion to knowledge-base update, showing every service and decision point it passes through:



This follows an evidence-first, human-in-the-loop operating model consistent with current AI SRE guidance.

8. Suggested Microservices



9. Database Design

Core entities and their fields, plus foreign-key relations across the incident lifecycle:

INCIDENT incident_id · title · description · status · severity · priority · owner · created_at · resolved_at · ai_summary · confidence_score	ALERT alert_id · source · service · metric · value · timestamp · incident_id (FK)
SERVICE service_id · name · owner · team · repository · runbook_url · dependency_graph	RCA rca_id · incident_id (FK) · hypothesis · confidence · evidence · created_at
RUNBOOK runbook_id · service · title · steps · automation_supported	POSTMORTEM postmortem_id · incident_id (FK) · summary · timeline · action_items · lessons
KNOWLEDGE_DOCUMENT document_id · title · source · embedding_id · tags	INCIDENT_EVENT event_id · incident_id (FK) · actor · type · description · timestamp

10. API Design

Incident APIs

POST /api/v1/incidents
GET /api/v1/incidents
GET /api/v1/incidents/{id}
PATCH /api/v1/incidents/{id}
DELETE /api/v1/incidents/{id}

Alert APIs

POST /api/v1/alerts
GET /api/v1/alerts

AI APIs

```
POST /api/v1/ai/investigate
POST /api/v1/ai/rca
POST /api/v1/ai/impact
POST /api/v1/ai/runbook
POST /api/v1/ai/postmortem
```

Automation APIs

```
POST /api/v1/actions/restart
POST /api/v1/actions/rollback
POST /api/v1/actions/scale
POST /api/v1/actions/verify
```

Knowledge APIs

```
POST /api/v1/knowledge/upload
POST /api/v1/knowledge/index
GET /api/v1/knowledge/search
```

11. Technology Stack

Layer	Technology
Backend	Java 21 + Spring Boot
AI	Spring AI
Agent Workflow	LangGraph4j or Spring AI Agents
Database	PostgreSQL
Vector DB	PGVector
Search	OpenSearch
Cache	Redis
Messaging	Kafka
Storage	MinIO
Security	Spring Security + OAuth2 + Keycloak
Workflow	Camunda 8
Frontend	React + Material UI
Deployment	Kubernetes / OpenShift
Observability	Prometheus + Grafana + OpenTelemetry

12. Minimum Production-Ready Version (MVP)

Phase 1 — Weeks 1–4

- Alert ingestion
- Incident creation
- Incident dashboard
- User authentication
- PostgreSQL persistence
- Redis cache
- Kafka event bus

Phase 2 — Weeks 5–8

- Spring AI integration
- RAG with PGVector
- AI incident summary
- AI severity prediction
- Similar incident retrieval
- Runbook recommendations

Phase 3 — Weeks 9–12

- Multi-agent orchestration
- Slack and Jira integration
- Approval workflow
- AI-generated timelines
- AI postmortem generation

Phase 4 — Enterprise

- Kubernetes automation
- GitOps integration
- Human approval gates
- Multi-tenant support
- Audit trail
- Cost analytics
- SLO/MTTR dashboards
- Autonomous remediation with policy guardrails

13. Production Package Structure (Java)

```
com.air
├─ api
├─ incident
├─ alert
├─ ai
│   ├── orchestrator
│   ├── agents
│   │   ├── investigation
│   │   ├── rca
│   │   ├── impact
│   │   ├── runbook
│   │   ├── automation
│   │   └─┬─ postmortem
│   ├── rag
│   └─ prompts
├─ workflow
├─ notification
├─ security
├─ audit
├─ knowledge
├─ common
└─ config
```

14. Competitive Positioning

Dimension	Traditional incident mgmt tools	Generic AIOps / alert noise reduction	AIR
Core function	Paging, escalation, status pages	Alert correlation and dedup	End-to-end investigation, diagnosis, and remediation
Role of AI	Absent or bolt-on summarization	Statistical clustering, limited reasoning	Multi-agent LLM reasoning over RAG-retrieved context
Remediation	Manual runbooks, human-executed	Not in scope	Agent-proposed automation, human-approved before execution
Governance	Basic RBAC	Limited	Dedicated policy engine, full audit of every AI and human action
Postmortems	Manually authored	Not in scope	AI-drafted from captured timeline, human-reviewed

15. Risks & Open Questions

- **LLM hallucination in RCA:** a confidently-worded incorrect root cause can misdirect an on-call engineer. Confidence scoring and mandatory evidence citation need to be enforced, not optional.
- **Autonomous action safety:** remediation actions (restart, rollback, scale) touch production. Blast-radius estimation and staged rollout of automation trust need to precede any expansion of the approval-free action set.
- **Data governance:** logs, traces, and incident data sent to an LLM may include customer or sensitive data. Redaction and data-residency policy need definition before Phase 2.
- **Multi-tenant isolation:** knowledge bases, runbooks, and RCA history must not leak across tenants sharing the platform.

- **Evaluation of agent accuracy:** there's no established benchmark yet for RCA correctness or postmortem quality — an internal eval harness against historical incidents is needed before enterprise rollout.
- **Trust and adoption:** on-call engineers may distrust or routinely override AI suggestions early on; UX must make override fast and frictionless while still capturing why, to improve the system.
- **Cost at scale:** vector search and LLM calls per incident add real-time inference cost; needs monitoring as incident volume grows.
- **Compliance:** audit trail retention and explainability requirements (SOC 2, industry-specific regulation) may constrain what agents can act on autonomously.

16. Glossary

RCA — Root Cause Analysis; the process of identifying the underlying cause of an incident.

MTTR — Mean Time to Resolution; the average time from incident detection to resolution.

Runbook — a predefined, structured procedure for diagnosing or remediating a known failure mode.

Agent Orchestrator — the component that sequences and coordinates specialized AI agents during an incident.

Policy Engine — the governance layer enforcing RBAC and approval requirements before automated actions execute.

Human-in-the-loop — an operating model where AI proposes actions but a human must approve state-changing steps.

RAG — Retrieval-Augmented Generation; grounding an LLM's output in retrieved documents (runbooks, past incidents) rather than model memory alone.

Blast Radius — the scope of systems, customers, or revenue affected by an incident.

Postmortem — a written retrospective on an incident: timeline, root cause, and follow-up actions.

Vector DB — a database indexing embeddings for semantic similarity search, used to retrieve relevant runbooks and past incidents.

SLO — Service Level Objective; a target reliability or performance threshold for a service.

Multi-tenant — architecture supporting multiple isolated customer organizations on shared infrastructure.
